

Advanced Algorithms: A* 8-Puzzle Solver

What I Built

I made an interactive command-line 8-puzzle solver that implemented the A* search algorithm in C++.

The puzzle is a sliding-tile game that consists of 8 numbered tiles and a blank space arranged on a 3 by 3 board. The puzzle is considered solved once the numbers are arranged in sequential order, reaching the configuration:

1	2	3
4	5	6
7	8	

The A* algorithm selects which puzzle state to explore using the evaluation function:

$$f(n) = g(n) + h(n)$$

where **$g(n)$** is the cost of reaching the current state and **$h(n)$** is an estimate of the remaining cost to reach the goal. This combination of path cost and heuristic estimate is the basis of A* search.

In my implementation, every tile movement has a cost of 1. The solver supports two heuristic functions:

- **Manhattan Distance**
- **Misplaced Tiles**

The program also provides functionality for generating random solvable puzzles, validating manually entered puzzles, displaying the solution path, and reporting search statistics.

The recorded search statistics are:

- States expanded
- States generated
- Maximum open-set size

Track B: Working Tool

For Track B, I used my A* implementation in a simple CLI program.

The program allows users to create their own 8-puzzle for the algorithm to solve and observe the resulting search.

The main functionality consists of:

- Solving a manually entered puzzle
- Generating a random solvable puzzle
- Selecting the heuristic used by A*
- Comparing the implemented heuristics
- Viewing the resulting solution path
- Viewing search statistics

Heuristics

The first heuristic implemented is **Manhattan Distance**. For each numbered tile, the heuristic calculates the absolute difference between the tiles current position and its goal position:

$$|\text{current row} - \text{goal row}| + |\text{current column} - \text{goal column}|$$

The distances for all numbered tiles are then summed.

The second heuristic is **Misplaced Tiles**, which counts the number of numbered tiles that are not currently in their goal positions.

These heuristics provide different amounts of information about the current puzzle state. Manhattan Distance considers how far each tile is from its destination, whereas Misplaced Tiles only considers whether each tile is in the correct position. Both are commonly used heuristic approaches for the 8-puzzle.

Heuristic Comparison

The program reports:

Metric	Manhattan Distance	Misplaced Tiles
Solution Moves	---	---
States Expanded	---	---
States Generated	---	---
Maximum Open Set	---	---

The actual values depend on the puzzle being tested.

This provides an empirical way of examining how the heuristic affects the search. In particular, the number of states expanded provides an indication of how much of the search space A^* needed to explore before reaching the goal.

Random Puzzle Generation

The random puzzle feature begins with the solved puzzle and performs a sequence of randomly selected legal moves.

This approach ensures that every generated configuration is reachable from the solved state and is therefore solvable.

The number of random moves does not represent the final solution length because the random process can undo previous moves.

Implementation

The open set used by A^* is implemented using a priority queue, ordering states according to their $f(n)$ value.

A hash-based set tracks states that have already been processed. This prevents the same board configuration from being expanded repeatedly.

The implementation also maintains parent relationships between states. When the goal is reached, these relationships are followed backwards from the goal to the starting state and then reversed to reconstruct the solution path.

What I Learned

Researching and implementing A* gave me a better understanding of how pathfinding algorithms can be applied to game development. Before implementing the algorithm, I understood A* primarily through the formula:

$$f(n) = g(n) + h(n)$$

I understood that the inclusion of $h(n)$ distinguished A* from algorithms such as Dijkstra's, but I did not initially appreciate how much the choice of heuristic could affect the search.

Implementing both Manhattan Distance and Misplaced Tiles in my CLI made this clearer. I found that while both heuristics estimate the remaining cost, Manhattan Distance provides more information by considering how far each tile is from its goal, whereas Misplaced Tiles only considers whether a tile is in the correct position. Comparing their search statistics helped me understand the practical value of choosing an appropriate heuristic, rather than viewing $h(n)$ as simply another part of the A* formula.

AI Usage (ChatGPT)

For this assignment I exclusively used ChatGPT for help with the initial setup and implementation of the application. This includes explaining and directing me to additional resources for understanding more specific concepts or just deepening my understanding of the A* algorithm. I also used it to generate a report skeleton based on the readme for my repository.

An instance of the AI being a bit unhelpful was when it initially recommended compiling the project using C++17. My implementation used `std::unordered_set::contains()`, which requires C++20, so the suggested compilation command was incompatible with my implementation. I identified this from the compiler error and corrected the compilation command to use C++20.

Another example was when AI suggested an example puzzle for testing that turned out to be unsolvable. My implementation correctly identified that the puzzle could not be solved through its solvability check, allowing me to catch the issue rather than assuming the suggested example was valid.

I trusted that the automated tests for the algorithm were correct whereas I was more involved and interested in understanding the core A* implementation.

References

P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

Tarodev, "Pathfinding - Understanding A* (A star)," *YouTube*, 16 November 2021. Available: <https://www.youtube.com/watch?v=i0x5fj4PqP4>

[A* algorithm Explained like you're a 5th Grader] Available: https://www.youtube.com/watch?v=qigPctCgR_k